Week 3 & 4: Deep Dive into JavaScript

DevPreneurs MERN Stack Internship Program

These two weeks are dedicated entirely to solidifying JavaScript fundamentals. A strong grasp of vanilla JavaScript is strictly required before transitioning into React.

Week 3: Core Mechanics & DOM Mastery

Day 1: Data Structures & Arrays

- **Study:** In-depth look at Objects and Arrays. Master array iteration methods: `map()`, `filter()`, `reduce()`, and `forEach()`.
- **Task:** Create a JavaScript file containing an array of mock user objects (name, role, active status). Write functions using higher-order array methods to filter active users, map their names into a new array, and calculate the total number of admins.

Day 2: Functions, Scope, & Closures

- **Study:** Function declarations vs. Arrow functions, lexical scope, block scope (let/const), and closures. Understand how variables are accessed in nested functions.
- **Task:** Build a functional counter using closures. Create a function that returns an object with methods to `increment`, `decrement`, and `reset` a private count variable that cannot be accessed directly from the global scope.

Day 3: Advanced DOM & Event Delegation

- **Study:** Event bubbling, event capturing, and event delegation. Creating, appending, and removing DOM nodes dynamically.
- **Task:** Build a dynamic list UI. Add a single event listener to the parent `<ul>` element (event delegation) to handle clicks on dynamically generated `<li>` elements, toggling a "completed" CSS class on them.

Day 4: Forms & Input Validation Fundamentals

- **Study:** Preventing default form submission (`e.preventDefault()`), reading form data (FormData object), and writing custom validation logic in JS.
- **Task:** Create a user registration form. Write a script to validate that the password is at least 8 characters long and matches a "confirm password" field before allowing the submission to proceed, showing inline error messages.

Day 5: Week 3 Mini-Project

- **Task:** Build a "Task Tracker" application. Users should be able to type a task into an input, add it to a list, mark items as complete, and delete items. All logic must use pure vanilla JavaScript manipulating the DOM, heavily utilizing arrays and event delegation.

Week 4: Asynchronous JavaScript & APIs

Day 1: Timers & The Event Loop

- **Study:** Synchronous vs. Asynchronous code, the Call Stack, the Event Loop, `setTimeout`, and `setInterval`.
- **Task:** Create a visual digital stopwatch on a webpage with Start, Stop, and Reset buttons. Ensure the timer formats the output as `MM:SS:MS` and does not glitch if the user clicks "Start" multiple times.

Day 2: Promises & Fetch API Fundamentals

- **Study:** Understanding Promises (Pending, Resolved, Rejected states), `.then()` and `.catch()` blocks. Making HTTP GET requests using the native Fetch API.
- **Task:** Fetch mock user data from a public API (like JSONPlaceholder). Parse the JSON response and dynamically render a grid of user profile cards on the DOM.

Day 3: Async / Await & Error Handling

- **Study:** Refactoring Promise chains into `async/await` syntax. Proper error handling using `try/catch/finally` blocks.
- **Task:** Rewrite Day 2's API fetch using `async/await`. Add logic to purposefully request a broken URL to trigger an error, and gracefully display a fallback error UI to the user instead of crashing the script.

Day 4: Browser Storage & Modules (ES6)

- **Study:** `localStorage` vs `sessionStorage`. ES6 module syntax (`import` and `export`) to split JavaScript into multiple manageable files.
- **Task:** Take the Task Tracker from Week 3 and save the array of tasks to `localStorage`. When the page reloads, retrieve the parsed JSON data and repopulate the UI so the tasks persist.

Day 5: Week 4 Mini-Project

- **Task:** Build a dynamic "Weather & Quote Dashboard".
- **Requirements:** Use Fetch/Async Await to pull a random quote from a public API on load. Allow the user to save their favorite quotes to `localStorage`. Structure your code modularly using ES6 imports/exports (e.g., separate files for API logic, UI rendering, and storage management).